

你已经在用 AI 写代码了。然后呢？



过去一个月，用 AI 写了超过 50% 代码的？



用过 gstack / superpowers / Cursor Rules 的？



曾经被 AI 写的代码坑过的？

工具越来越多、AI 越来越强——但你的「掌控感」并没有对应提升。

执行已被商品化

AI 几分钟就能给你一份 Harness 清单

• • • Claude Code – 一套最佳实践配置

- ✓ CLAUDE.md (项目约定 + 代码风格)
- ✓ rules/ 分类规则 (命名 / 测试 / 提交)
- ✓ hooks/ (pre-commit / lint / 格式化)
- ✓ skills/ (review / ship / refactor)
- ✓ MCP server 接入清单
- ✓ 子代理与并行编排建议
- 还附带了「为什么这样配」的解释

30 seconds.

Free.

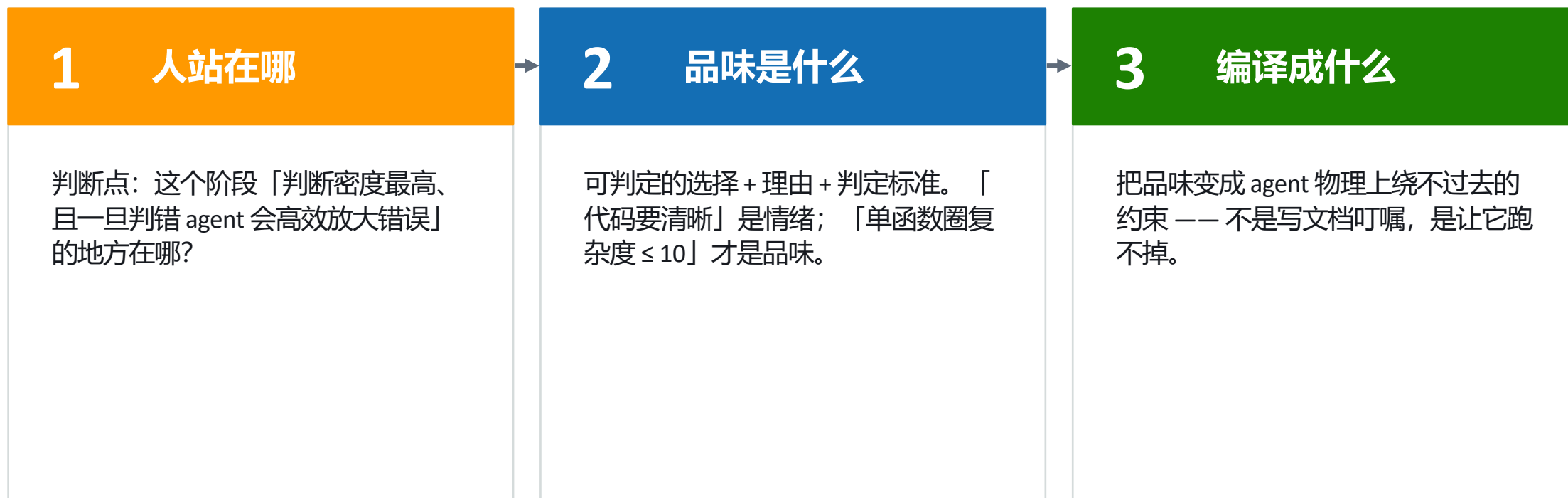
Anyone can get this.

所以你的 IP 不在此处 ↑

执行（配置文件、工具配置、流程清单）已经是商品。
那不是商品的，是什么？

这门课只教 AI 给不了的那部分

在每个研发阶段，人必须站在哪儿、用什么判断、把什么品味编译成 agent 跑不掉的约束。



这是北极星：三天里每个阶段（架构 / 设计 / 编码 / 验证 / 运维）都做一次这个动作。

AgentOS = 让 AI Agent 在你的项目中受控运行、持续进化的操作系统

受控运行

agent 按你定义的流程走，该停的地方停、该检查的地方检查，不跳步。

持续进化

系统越用越精，不是越用越臃肿 —— 越用越短、覆盖越来越广。

三天后你带走的 deliverable :

my-agentos/

- | knowledge/ 领域知识 + 经验 (大脑)
- | governance/ 三层治理：原则/规则/门禁
- | engine/ Delivery Engine (手脚)
- | eval/ 行为契约自测
- | hooks/ skills/ 原语
- | corrections.log 进化证据

错题本是杠铃，AgentOS 是肌肉

你去健身房不是为了举杠铃，是为了长肌肉。
错题本只是练手素材；你带走的是套在它外面、
能挂到任何项目上的 AgentOS。 **这是你的 IP。**

AgentOS 的两个模块 + 一层原语



Knowledge = 大脑

知道什么、记住什么、约束什么、如何进化

Delivery Engine = 手脚

按什么流程做、在哪检查、什么时候停

Eval = 自我意识

我还在正轨吗？（融入验证阶段）

Harness = 基座

Agent 运行时基础设施，被模块组合使用

Knowledge = 大脑：知道、记住、约束、进化

组件	一句话	类比
DDD	项目的结构化领域知识	团队 wiki, 但 agent 可读
Memory	跨 session 的经验积累	人的记忆, 分短期 / 长期
Governance	行为约束的三层治理	社会的道德 / 法律 / 执法
Distillation	做减法的进化机制	免疫系统: 记住病原体, 丢掉每次感冒的细节
Day 1 下午 搭骨架 Day 2 全天 实弹填充 Day 3 上午 做蒸馏		

Delivery Engine = 手脚：不可跳步的执行轨道

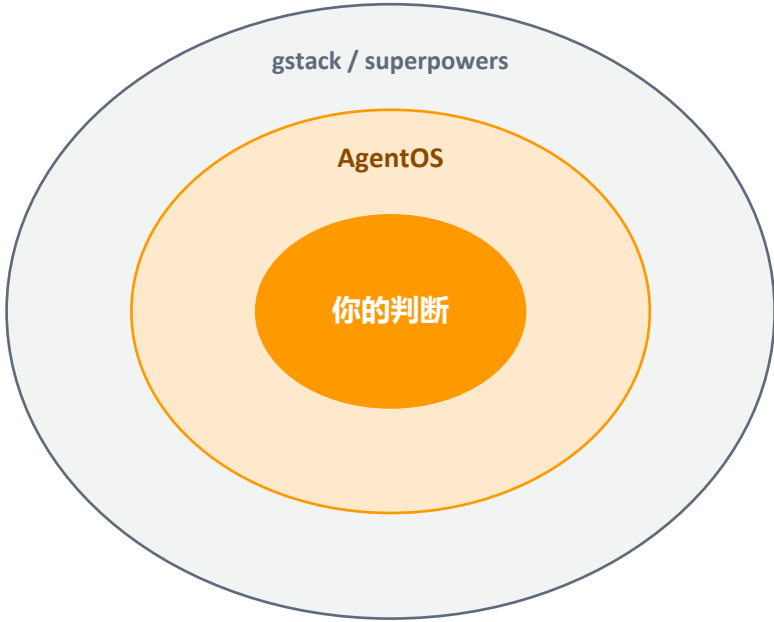
组件	一句话	解决什么问题
Stages	有序执行阶段	agent 会乱序做事
Gates	阶段间硬隔离	agent 会跳步（「我觉得差不多了」）
Profiles	任务类型 → 路径	一行 bugfix 不需要走全流程
State	执行位置记录	中断后 agent 不知道自己在哪

橙色 = Gate（这步没过不许进下一步） · Day 2 上午设计，Day 2 下午跑一遍



AgentOS 不替代 gstack —— 它编排 gstack

	gstack / superpowers 等	你的 AgentOS
编码的是	别人的品味	你的判断
知道你的	通用最佳实践	架构红线、验证标准、成本底线
会进化吗	作者更新时进化	你每次纠正时进化
能替代吗	不能替代 AgentOS	可以编排它们



gstack 是 App Store 里的 app，AgentOS 是操作系统。

你可以装 gstack 的 /review、/ship —— 但它们在你的 Engine 里跑、受你的 Gate 约束、产出写回你的 Knowledge。谁说了算？你说了算。

两个模块不是拼在一起，是相互增强

Knowledge 提供上下文



Engine 产出更精准



REFLECT 写回经验



Knowledge 更丰富



Engine 下次更好 → Corrections 减少



Governance 蒸馏 → Principles 更锋利
↓ (回到顶部)

复合飞轮

每转一圈，系统变强一点 (Compound)

可观测

corrections 数量递减 = 飞轮在转

终态

不是「零错误」，而是「新类型错误也能首次处理对」

三天里你会亲手转第一圈：Day 1 搭 Knowledge, Day 2 跑 Engine 并写回, Day 3 做蒸馏。

好的 AgentOS 越用越短，不是越用越长

反直觉：规则不是越多越好。证据——

- Anthropic: 「臃肿的 CLAUDE.md 会让 Claude 忽略你的指令」
- Princeton Reflexion: 反思条目 cap 在 3 条，更多反而降质
- SwarmAI 实战: 27 条 corrections, 同一偏差重复 4 次，规则没用

正确方向：蒸馏

- 多条 rules 同源 → 吸收为一条 principle
- Gate 已机械覆盖 → 对应 rule 退休
- Gate 30 天未触发 → 考虑毕业

✅ 健康	❌ 生病
文件变短 + 质量提升	文件持续膨胀
Gate 触发趋零	每个新错都需要新 gate
新错首次处理对	同一偏差换皮重复

规则只能枚举症状，症状是无限的。原则覆盖一整类。

Principles → Rules → Gates

PRINCIPLES (方向)

3-5 条, 覆盖类 ~70-80%

↓ 上层不够才下沉

RULES (指导)

有限 / 可追溯 / 可过期 ~85-90%

↓ 上层不够才下沉

GATES (强制)

最少, 代码可检测 ~99%

进化方向 ↑ 让高层越来越强, 底层触发越来越少

Principles

数量少 (3-5), 每条覆盖一整类失败

Rules

link to principle (追溯不到 → 候选删); 有毕业条件

Gates

最少化、机械化, 只在 3+ 次失败后才加

三层共存

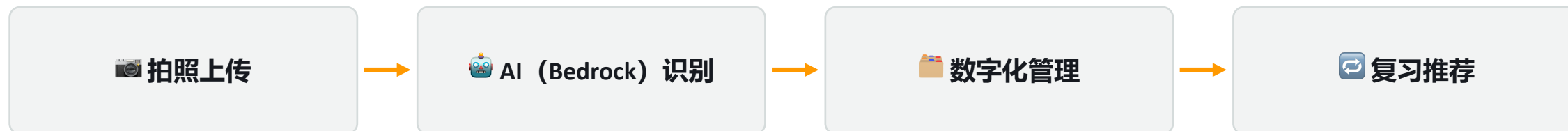
不是三选一。组合 ≈ 99.5%+ 正确行为

三天干什么



一句话：Day 1 造壳，Day 2 填肉，Day 3 炼金。

统一载体 + 个人判断



统一（载体）

- 代码
- 需求
- 技术栈
- 架构

产出可比、环境稳定、互评可行

个人（判断）

- principles
- rules
- gate 放置
- 验证标准
- 蒸馏选择 你的IP

同一个错题本，不同的 AgentOS —— 从 Day 2 起收敛到「拍照识别」单功能做纵深。

你的工作台

你的电脑 (浏览器)



AWS Workshop (每人独立环境)

- VSCode Server
- Claude Code CLI



Amazon Bedrock (错题识别)

Token 由环境提供

启动 Checklist

- ✓ 环境: AWS Workshop, 每人一个独立环境
- ✓ 工具: VSCode Server + Claude Code CLI
- ✓ 能力: Bedrock 访问 (识别用), Token 已配好
- ✓ 课前预习: Claude Code 功能速查 (已发)
- ✓ 现在: 打开终端, 确认 `claude --version` 能跑

后面做 loop 实验会烧 token —— 这是课程内容的一部分, 不要心疼, Day 3 讲成本治理时会回来算这笔账。

READY?

开始构建你的 AgentOS

• • • terminal

```
$ mkdir my-agentos && cd my-agentos && git init  
Initialized empty Git repository ...
```

NEXT ►

02

Harness 基座速览 (60 min)

Your OS starts now.

接下来: Harness 基座速览 → 下午 Knowledge 模块搭建。三天后这个目录就是你的 deliverable。